Cognitive Robotics Lab – 4

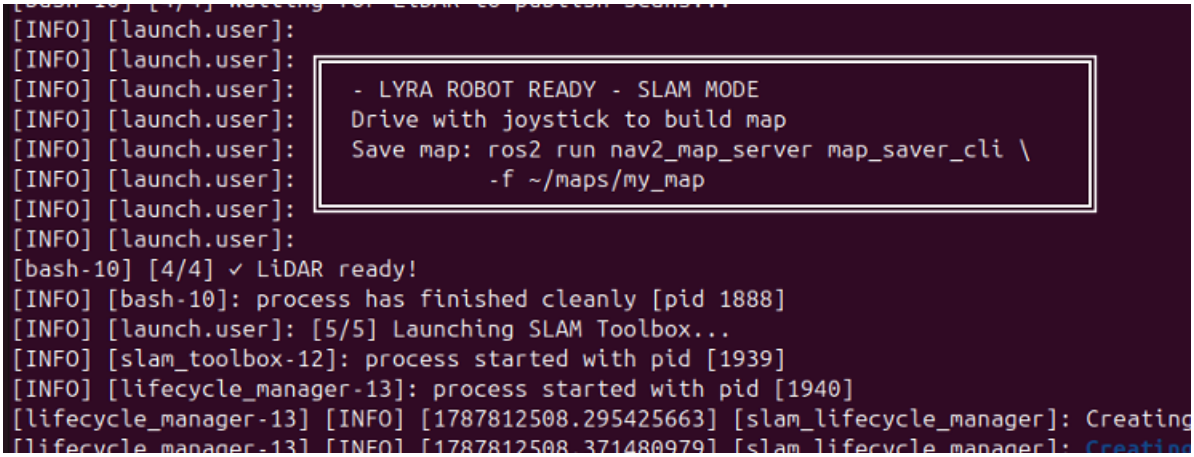
23BRS1177

Videos: https://github.com/Prayagraj27/Cognitive-Robotics-Lab

Traffic Sign Detection and Movement

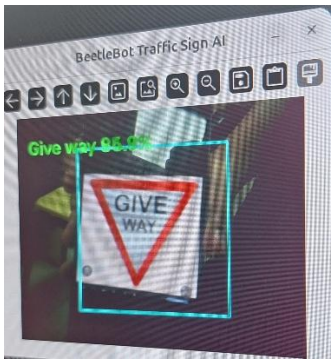
Signs in the model(dataset from Kaggle): Give way, Axle weight limit, Road hump, No entry, One-way traffic, One-way traffic , No vehicles in both directions , No entry for cycles , No entry for goods vehicles , No entry for pedestrians , No entry for bullock carts , No entry for hand carts , No entry for motor vehicles , Height limit , Weight limit , Length limit , No left turn , No right turn , No overtaking , Maximum speed limit (90 km/h) , Maximum speed limit (110 km/h) , Horn prohibited , No parking , No stopping , Turn left , Turn right , Steep descent , Steep ascent , Narrow road , Narrow bridge , Unprotected quay , Dip , Loose gravel , Falling rocks , Cattle , Crossroads , Side road junction , Side road junction , Oblique side road junction , T-junction , Y-junction , Staggered side road junction , Staggered side road junction , Roundabout , Guarded level crossing ahead , One-way traffic , First aid post , Unguarded level crossing ahead , Level crossing countdown marker , Level crossing countdown marker , Telephone , Level crossing countdown marker , Level crossing countdown marker , Parking , Bus stop , Filling station , Hotel , Restaurant , Refreshments

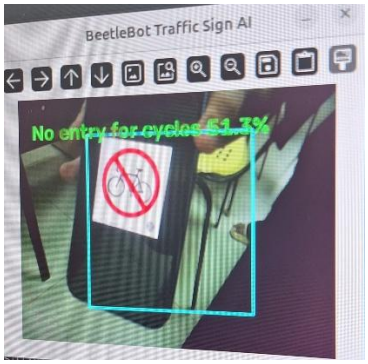
Outputs

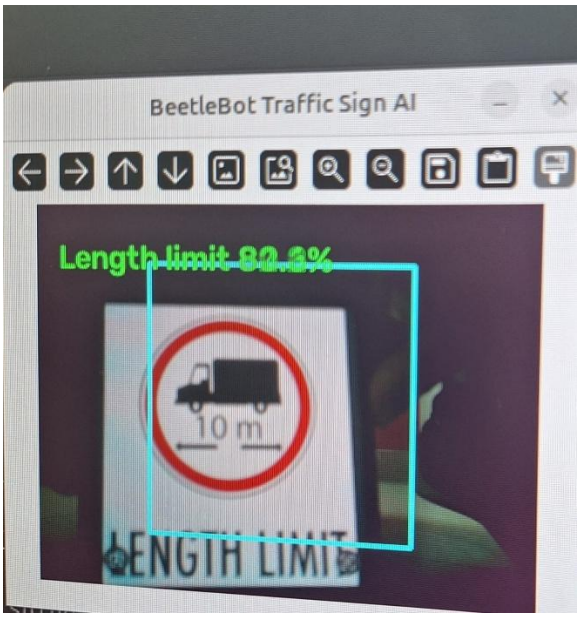


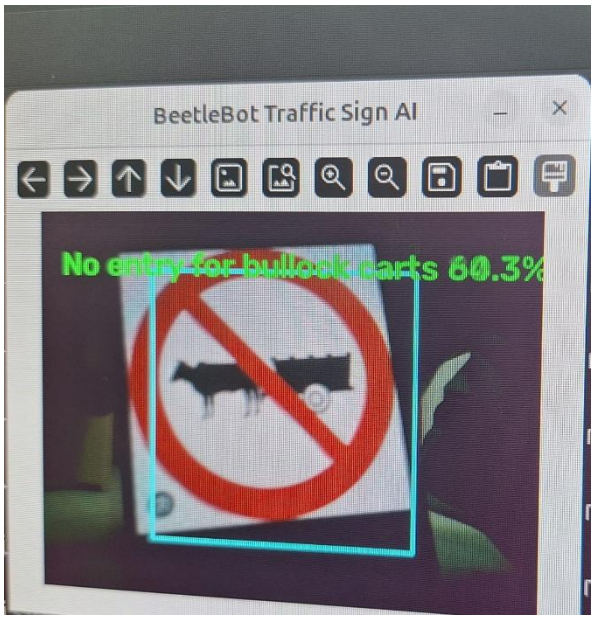
```
rclpy._rclpy_pybind11.RCLError: failed to shutdown: rcl_shutdown already called on the giv
n context, at ./src/rcl/init.c:333
(beetlebot_ai) student@jazzy-jalisco:~/beetlebot_ai$ python3 traffic_sign_camera.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1787816032.353733 1255132 port.cc:153] oneDNN custom operations are on. You ma
 see slightly different numerical results due to floating-point round-off errors from diff
rent computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_
PTS=0`.
I0000 00:00:1787816032.353934 1255132 cudart_stub.cc:31] Could not find cuda drivers on yo
r machine, GPU will not be used.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1787816033.085239 1255132 port.cc:153] oneDNN custom operations are on. You ma
 see slightly different numerical results due to floating-point round-off errors from diff
rent computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_
PTS=0`.
I0000 00:00:1787816033.085477 1255132 cudart_stub.cc:31] Could not find cuda drivers on yo
r machine, GPU will not be used.
E0000 00:00:1787816033.237885 1255132 cuda_platform.cc:52] failed call to cuInit: INTERNAL
 CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=wayland to run or
Wayland anyway.
QFontDatabase: Cannot find font directory /home/student/beetlebot_ai/.venv/lib/python3.12/
ite-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for
xample) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/student/beetlebot_ai/.venv/lib/python3.12/
ite-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for
xample) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/student/beetlebot_ai/.venv/lib/python3.12/
ite-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for
xample) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/student/beetlebot_ai/.venv/lib/python3.12/
ite-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for
xample) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/student/beetlebot_ai/.venv/lib/python3.12/
ite-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-fonts.github.io/ for
xample) or switch to fontconfig.
Killed
(beetlebot_ai) student@jazzy-jalisco:~/beetlebot_ai$ python3 traffic_sign_camera.py
```

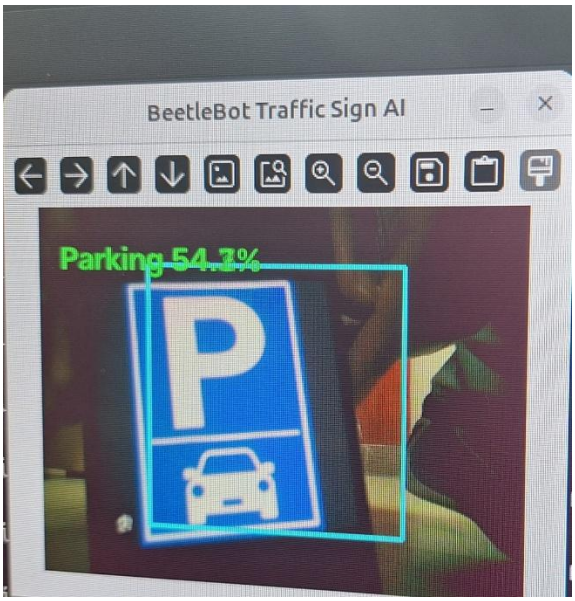


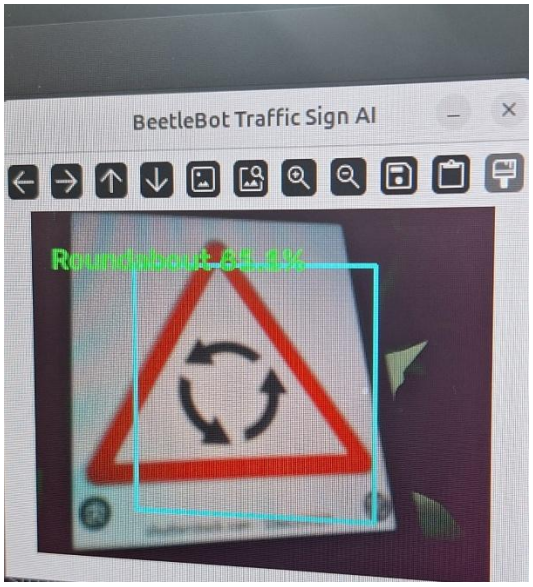





BeetleBot Traffic Sign AI
Length limit 80.3%
10 m
LENGTH LIMIT


BeetleBot Traffic Sign AI
No entry for bullock carts 60.3%


BeetleBot Traffic Sign AI
Parking 54.1%
P


BeetleBot Traffic Sign AI
Roundabout 85.4%

CODE:

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
import csv
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
import tensorflow as tf
import numpy as np
import cv2

MODEL_PATH = "/home/student/beetlebot_ai/traffic_sign_model.h5"
CSV_PATH = "/home/student/beetlebot_ai/traffic_sign.csv"
CAMERA_TOPIC = "/pi_camera/image_raw"
CMD_VEL_TOPIC = "/cmd_vel"

FORWARD_SPEED = 0.2
TARGET_STOP_SIGN = "no bicycles"

csv_names = {}
with open(CSV_PATH, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        class_id = int(row["ClassId"])
```

```
        csv_names[class_id] = row["Name"]

folder_names = sorted([str(i) for i in range(59) if i != 39])

model_index_to_class_id = {
    index: int(folder_name)
    for index, folder_name in enumerate(folder_names)
}

model = tf.keras.models.load_model(MODEL_PATH, compile=False)
IMG_SIZE = model.input_shape[1]

class TrafficSignDetector(Node):
    def __init__(self):
        super().__init__("traffic_sign_detector", enable_rosout=False)
        self.bridge = CvBridge()

        self.subscription = self.create_subscription(
            Image, CAMERA_TOPIC, self.image_callback, 10
        )
        self.cmd_vel_pub = self.create_publisher(Twist, CMD_VEL_TOPIC, 10)

    def image_callback(self, msg):
        try:
            frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
            h, w = frame.shape[:2]
```

```
crop_size = int(min(h, w) * 0.70)
x1 = (w - crop_size) // 2
y1 = (h - crop_size) // 2
x2 = x1 + crop_size
y2 = y1 + crop_size

sign_crop = frame[y1:y2, x1:x2]
sign_crop = cv2.cvtColor(sign_crop, cv2.COLOR_BGR2RGB)

img = cv2.resize(sign_crop, (IMG_SIZE, IMG_SIZE))
img = img.astype(np.float32)
img = np.expand_dims(img, axis=0)

probabilities = model.predict(img, verbose=0)[0]
model_index = int(np.argmax(probabilities))
confidence = float(probabilities[model_index])

class_id = model_index_to_class_id.get(model_index)
sign_name = csv_names.get(class_id, f"Class {class_id}")

cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 255, 0), 2)

twist = Twist()

if confidence >= 0.50:
```

```
            text = f"{sign_name}{confidence * 100:.1f}%"
            cv2.putText(frame, text, (15, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.70, (0, 255, 0), 2, cv2.LINE_AA)

            if TARGET_STOP_SIGN.lower() in sign_name.lower():
                twist.linear.x = 0.0
                cv2.putText(frame, "STATUS: STOPPED", (15, 80), cv2.FONT_HERSHEY_SIMPLEX, 0.70, (0, 0, 255), 2, cv2.LINE_AA)
            else:
                twist.linear.x = FORWARD_SPEED
                cv2.putText(frame, "STATUS: MOVING", (15, 80), cv2.FONT_HERSHEY_SIMPLEX, 0.70, (255, 255, 0), 2, cv2.LINE_AA)
        else:
            text = f"Low confidence {confidence * 100:.1f}%"
            cv2.putText(frame, text, (15, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.70, (0, 0, 255), 2, cv2.LINE_AA)

            twist.linear.x = FORWARD_SPEED
            cv2.putText(frame, "STATUS: MOVING", (15, 80), cv2.FONT_HERSHEY_SIMPLEX, 0.70, (255, 255, 0), 2, cv2.LINE_AA)

        self.cmd_vel_pub.publish(twist)

        cv2.imshow("BeetleBot Traffic Sign AI", frame)
        cv2.waitKey(1)

    except Exception as e:
```

```
            print(f"Error in image callback: {e}")


def main():
    rclpy.init()
    node = TrafficSignDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        stop_twist = Twist()
        stop_twist.linear.x = 0.0
        node.cmd_vel_pub.publish(stop_twist)


        node.destroy_node()
        rclpy.shutdown()
        cv2.destroyAllWindows()


if __name__ == "__main__":
    main()
```